

## Edge cases (fix a flaky test) — Hints

---

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper property checks), **Advanced** (flakiness that isn't caused by wording at all). Read Basic first even if you already know the pattern — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 1 — The idea, and the exact pieces

---

### Basic Version

Start with a broken test like this: `assert response == "Your order has shipped."` Run it a few times against a real LLM call — sometimes it passes, sometimes it fails, even though the model isn't actually getting anything wrong. It's just wording the same correct answer differently each time.

The fix isn't to make the check looser in a vague way — it's to figure out exactly *what* made that answer correct, and check for that specific thing instead of the exact words.

Ask yourself: what's the one fact or property that HAD to be true for that answer to count as correct? Write that down first.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

A test that sometimes passes and sometimes fails on the *same underlying behavior* is worse than a test that always fails — a reliably failing test at least gets investigated once; a flaky one gets a shrug and a re-run, and eventually gets ignored entirely, at which point it's providing zero protection while still looking like it's part of your safety net.

The fix is always the same shape: replace an exact-match assertion with a **property** assertion — something that's true of every correct answer, regardless of exact wording. Common properties:

- **Keyword/substring check:** `"shipped"` in `response.lower()` — the `.lower()` matters, since capitalization shouldn't be part of what makes an answer correct.
- **Valid JSON check:** wrap `json.loads(response)` in a `try/except json.JSONDecodeError`, and assert no exception was raised — this checks *structure*, not exact content.
- **Length bound:** `len(response.split()) < 100` — useful when a rule genuinely is "must be under N words."
- **Any-of-several-phrasings:** `any(phrase in response.lower() for phrase in [...])` — for when there's more than one genuinely correct way to say the same thing.

Take one specific flaky assertion (from your own tests, or a made-up one like `assert response == "Your order has shipped."`) and write down, in English, the property that actually matters, before writing code.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Advanced Version

Not every flaky test is flaky for the reason this exercise starts with. Wording variation is one cause, but there are others that look identical from the outside — a test that "sometimes fails" — and need a completely different fix:

- **Real randomness in the model call itself.** If `temperature` isn't set (or is set high), the same prompt can genuinely produce different outputs on different runs, independent of wording choice. For anything you want *reproducible* (not just "correct in spirit"), setting `temperature=0` on the call under test reduces this source of variance — though it won't make an LLM call fully deterministic the way a pure function is.
- **Network flakiness, not model flakiness.** A timeout, a rate limit, a transient 503 — these have nothing to do with whether the answer was good, but they show up as the exact same kind of "sometimes red, sometimes green" test. Retrying blindly hides this; it doesn't fix it. The real fix is separating "did the call fail for infrastructure reasons" from "did the call succeed but score badly" — don't let a network retry and a wording fix live in the same `except` block.
- **A genuinely unreliable property, not a genuinely reliable one.** If you rewrite a test to check `"shipped" in response.lower()` but the model sometimes correctly says "on its way" instead, your *fix* just introduced new flakiness from an incomplete keyword list — not fixed the old kind.

The real design question: when a test fails intermittently, how do you tell which of these three you're looking at, instead of guessing? One answer — run the exact same input through the exact same test **N times in a row** (say, 10) and look at the pass rate, not just one pass or fail:

- Pass rate near 100%, occasional single failures → likely wording variance; tighten the property.
- Pass rate that swings a lot, or clusters of failures right after a timeout in the logs → likely infrastructure, not correctness; fix retry/timeout handling separately, don't touch the assertion.
- Consistent partial failure rate (like always failing ~20% of the time) on a property you thought was solid → your property itself is incomplete; the keyword list needs to grow, or you need a judge instead of a keyword check.

Sketch a small script that calls the same test input 10 times and reports the pass rate, before checking Hint 2.

**Difference between Basic, Intermediate, and Advanced:** Basic and Intermediate both assume the flakiness comes from exact-match wording variance, and fix it with a property check — correct for the specific broken test given in this exercise. Advanced steps back and asks: what if the flakiness *isn't* about wording at all? It covers 2 other real causes (sampling randomness, network transience)

that look identical from the outside but need different fixes, plus a repeated-run technique for telling the 3 causes apart before you decide which fix applies.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 2 — The plan, and almost the whole thing

---

### Basic Version

```
broken test:
    assert response == "Your order has shipped."

fixed test:
    response = call the model
    assert "shipped" is somewhere in response, ignoring capitalization
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# llm_output_testing_practice.py – Edge cases section
def test_shipping_reply():
    response = call_model("Has my order shipped?")
    assert "shipped" in response.lower()
```

Add one more assert for a reasonable length limit yourself.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

```
broken test (flaky – fails on a re-run with an equally correct answer):
def test_shipping_reply():
    response = call_model("Has my order shipped?")
    assert response == "Your order has shipped."
```

```
fixed test (checks the property that actually matters):
def test_shipping_reply():
    response = call_model("Has my order shipped?")
    assert "shipped" in response.lower()
    assert len(response.split()) < 60
```

```
# llm_output_testing_practice.py – Edge cases section
def test_shipping_reply() -> None:
    response = call_model("Has my order shipped?")
    # property, not exact wording: the key fact must be present
    assert "shipped" in response.lower()
    # property: the answer should be a short, direct reply, not a ramble
    assert len(response.split()) < 60
```

What's missing: a version that covers several genuinely correct phrasings ("shipped," "on its way," "dispatched") instead of just one keyword. Write it yourself before moving to Advanced.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Advanced Version

```
function pass_rate_over_n_runs(test_input, n):
    results = []
    for i in range(n):
        response = call_model(test_input, temperature=0)
        passed = check the property (e.g. "shipped" in response.lower())
        results.append(passed)
    return count of True in results / n
```

use this to diagnose a flaky test BEFORE deciding how to fix it:  
 rate = pass\_rate\_over\_n\_runs("Has my order shipped?", 10)  
 if rate is near 1.0 with one-off misses: likely a wording gap, widen the phrase  
 if rate swings a lot / correlates with timeouts in logs: likely infrastructure  
 if rate is a stable, repeatable fraction below 1.0: the property itself is in

Here's almost the whole thing — fill in the diagnosis logic yourself:

```
# llm_output_testing_practice.py – Edge cases section
def pass_rate_over_n_runs(test_input: str, n: int = 10) -> float:
    passed_count = 0
    for _ in range(n):
        response = call_model(test_input, temperature=0)
        acceptable_phrases = ["shipped", "on its way", "sent out", "dispatched"]
        if any(phrase in response.lower() for phrase in acceptable_phrases):
            passed_count += 1
    return passed_count / n

def test_shipping_reply_is_reliable() -> None:
    rate = pass_rate_over_n_runs("Has my order shipped?", n=10)
    # your turn: what pass rate should you require here, and why not exactly 1.0?
    ...
```

Decide (and justify) your own pass-rate bar, then compare all 3 of your finished versions against the [Solution](#).

**Difference between Basic, Intermediate, and Advanced:** Basic and Intermediate both fix the one broken assertion given in this exercise, by widening it from exact match to a property. Advanced adds a diagnostic tool — running the same input N times and looking at the pass rate — that tells

you *whether* an assertion rewrite is even the right fix, versus a retry/timeout problem or an incomplete keyword list, before you spend time rewriting the wrong thing.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)